Theory of Computation

Lecture 11

$$\overline{A_{\mathrm{TM}}} = \{\langle M, w\rangle : M \text{ does NOT accept } w\}$$

So R behaves like

- If M does not accept w, then R accepts (M, w).
- If M accepts w, then R may reject or loop indefinitely on (M, w).

Now consider another Turing Machine D
such that on input M, run R on (M, w_M).

$$\overline{A_{\mathrm{TM}}} = \{\langle M, w\rangle : M \text{ does NOT accept } w\}$$

So R behaves like

- If M does not accept w, then R accepts $\langle M, w\rangle$.
- If M accepts w, then R may reject or loop indefinitely on $\langle M, w\rangle$.

$$D \text{ accepts } M \iff R \text{ accepts } \langle M, \langle M\rangle\rangle.$$

$$\overline{A_{\mathrm{TM}}} = \{\langle M, w\rangle : M \text{ does NOT accept } w\}$$

So R behaves like

- If M does not accept w, then R accepts $\langle M, w\rangle$.
- If M accepts w, then R may reject or loop indefinitely on $\langle M, w\rangle$.

$$D \text{ accepts } M \iff M \text{ does not accept } \langle M\rangle.$$

Universality of CFL

Consider the language of universal CFGs:

$$\mathrm{ALL}_{\mathrm{CFL}} = \{\langle G \rangle : \mathscr{L}(G) = \Sigma^*\}.$$

Is $\mathrm{ALL}_{\mathrm{CFG}} \in \mathrm{R}(\Sigma)$?

Refutation is easy. Proof is hard!

Universality of CFL

Consider the language of universal CFGs:

$$\mathrm{ALL}_{\mathrm{CFL}} = \{\langle G \rangle : \mathscr{L}(G) = \Sigma^*\}.$$

We will use the fact that $\overline{A_{\mathrm{TM}}} \notin \mathrm{RE}(\Sigma)$.

Universality of CFL

Let M be a Turing Machine.

An accepting computation of M on w is $C_0\#C_1\#\cdots\#C_k$ where C_0 is the start configuration of M on w.

Universality of CFL

Let M be a Turing Machine.

An accepting computation of M on w is $C_0\#C_1\#\cdots\#C_k$ where C_0 is the start configuration of M on w.

Let $\mathrm{VAL}_{M,w} = \{x : x$ is a valid accepting computation of M on $w\}$

Is it context-free?

Universality of CFL

Let M be a Turing Machine.

An accepting computation of M on w is $C_0\#C_1\#\cdots\#C_k$ where C_0 is the start configuration of M on w.

Let $\mathrm{VAL}_{M,w} = \{x : x$ is a valid accepting computation of M on $w\}$

For all Turing Machines M and words w, $\overline{\mathrm{VAL}_{M,w}}$ is context-free.

Universality of CFL

For all Turing Machines M and words w, $\overline{\mathrm{VAL}_{M,w}}$ is context-free.

A string $x \notin \mathrm{VAL}_{M,w}$ if:

- There is a syntax error in the encoding
- It does not begin with the correct start configuration, or
- The last configuration is not accepting, or
- Some adjacent configurations do not match a legal transition.

Universality of CFL

For all Turing Machines M and words w, $\overline{\mathrm{VAL}_{M,w}}$ is context-free.

A string $x \notin \mathrm{VAL}_{M,w}$ if:

- There is a syntax error in the encoding
- It does not begin with the correct start configuration, or
- The last configuration is not accepting, or
- Some adjacent configurations do not match a legal transition.

The first three are regular. The last one can be checked by a PDA.

Universality of CFL

Observe that $\overline{\mathrm{VAL}_{M,w}} = \Sigma^* \iff \mathrm{VAL}_{M,w} = \emptyset$

Universality of CFL

Observe that $\overline{\mathrm{VAL}_{M,w}} = \Sigma^* \iff \mathrm{VAL}_{M,w} = \emptyset$

That is, $\overline{\mathrm{VAL}_{M,w}} \in \mathrm{ALL}_{\mathrm{CFL}} \iff$ there is no accepting run of M on w.

Universality of CFL

Observe that $\overline{\mathrm{VAL}_{M,w}} = \Sigma^* \iff \mathrm{VAL}_{M,w} = \emptyset$

That is, $\overline{\mathrm{VAL}_{M,w}} \in \mathrm{ALL}_{\mathrm{CFL}} \iff \langle M, w \rangle \in A_{\mathrm{TM}}$.

Universality of CFL

Observe that $\overline{\mathrm{VAL}_{M,w}} = \Sigma^* \iff \mathrm{VAL}_{M,w} = \emptyset$

That is, $\overline{\mathrm{VAL}_{M,w}} \in \mathrm{ALL}_{\mathrm{CFL}} \iff \langle M, w \rangle \notin A_{\mathrm{TM}}$.

For sake of contradiction, suppose we had a decider for $\mathrm{ALL}_{\mathrm{CFL}}$, then A_{TM} is also decidable.

Universality of CFL

Observe that $\overline{\mathrm{VAL}_{M,w}} = \Sigma^* \iff \mathrm{VAL}_{M,w} = \emptyset$

That is, $\overline{\mathrm{VAL}_{M,w}} \in \mathrm{ALL}_{\mathrm{CFL}} \iff \langle M, w \rangle \notin A_{\mathrm{TM}}$.

For sake of contradiction, suppose we had a decider for $\mathrm{ALL}_{\mathrm{CFL}}$, then A_{TM} is also decidable.

Hence universality of CFL is not decidable.

Proof by Reduction

Any arbitrary instance of A_{TM} can be reduced to a particular instance of $\mathrm{ALL}_{\mathrm{CFL}}$.

That is, A_{TM} can be reduced to $\mathrm{ALL}_{\mathrm{CFL}}$.

So $\mathrm{ALL}_{\mathrm{CFL}}$ is at least as hard as A_{TM}.

Two Contradiction Techniques

Diagonalisation

A_{TM}

Reduction

$\mathrm{ALL}_{\mathrm{CFL}}$

From Assessment 2:

Prove that the Halting problem is not recursive.

The Halting Problem

$$\mathrm{HALT}_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ halts on } w\}$$

For sake of contradiction, suppose H decided $\mathrm{HALT}_{\mathrm{TM}}$.

The Halting Problem

$$\mathrm{HALT}_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ halts on } w\}$$

For sake of contradiction, suppose H decided $\mathrm{HALT}_{\mathrm{TM}}$.

Then construct a machine S such that:

Run $H\left(\langle M, w\rangle\right)$

The Halting Problem

$$\mathrm{HALT}_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ halts on } w\}$$

For sake of contradiction, suppose H decided $\mathrm{HALT}_{\mathrm{TM}}$.

Then construct a machine S such that:

Run $H\left(\langle M, w\rangle\right)$, if rejected then reject. If halts then simulate M on w.

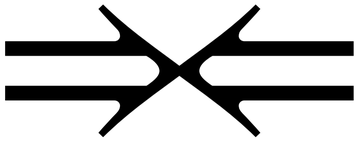
If accepted then accept, else reject.

The Halting Problem

$$\mathrm{HALT}_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ halts on } w\}$$

For sake of contradiction, suppose H decided $\mathrm{HALT}_{\mathrm{TM}}$.

We have a machine S that decides A_{TM}.



Turing's Proof

$$\mathrm{HALT}_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ halts on } w\}$$

For sake of contradiction, suppose H decided $\mathrm{HALT}_{\mathrm{TM}}$.

Construct a machine D such that:

Turing's Proof

$$\mathrm{HALT}_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ halts on } w\}$$

For sake of contradiction, suppose H decided $\mathrm{HALT}_{\mathrm{TM}}$.

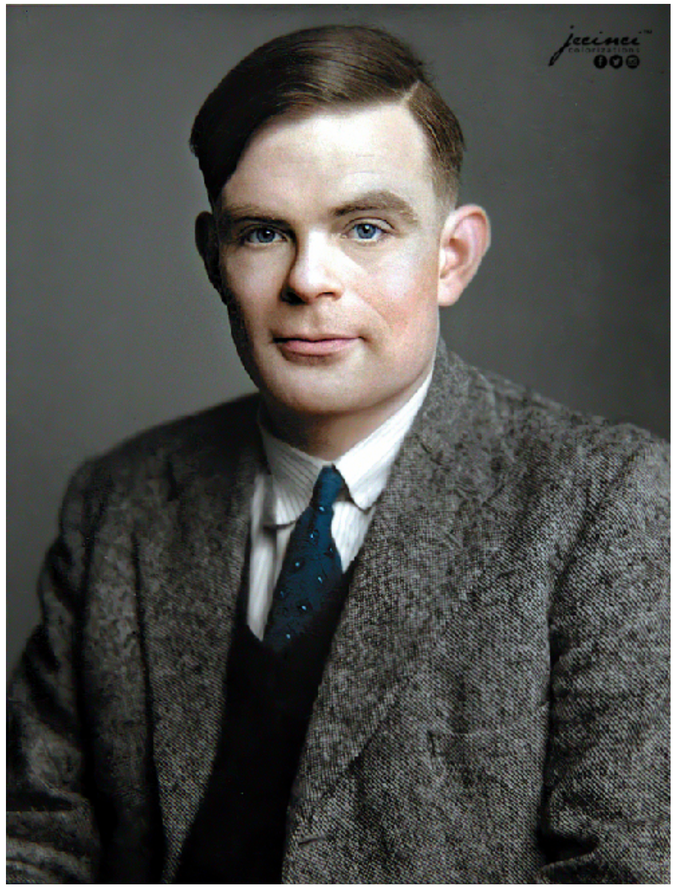
Construct a machine D such that:

Run $H\left(\langle M, \langle M\rangle\rangle\right)$, if accepted then loop forever. If rejected then accept.

What is $D\left(\langle D\rangle\right)$?

What is a Computable Number?

$$0.d_1 d_2 d_3 \cdots$$



Alan Turing

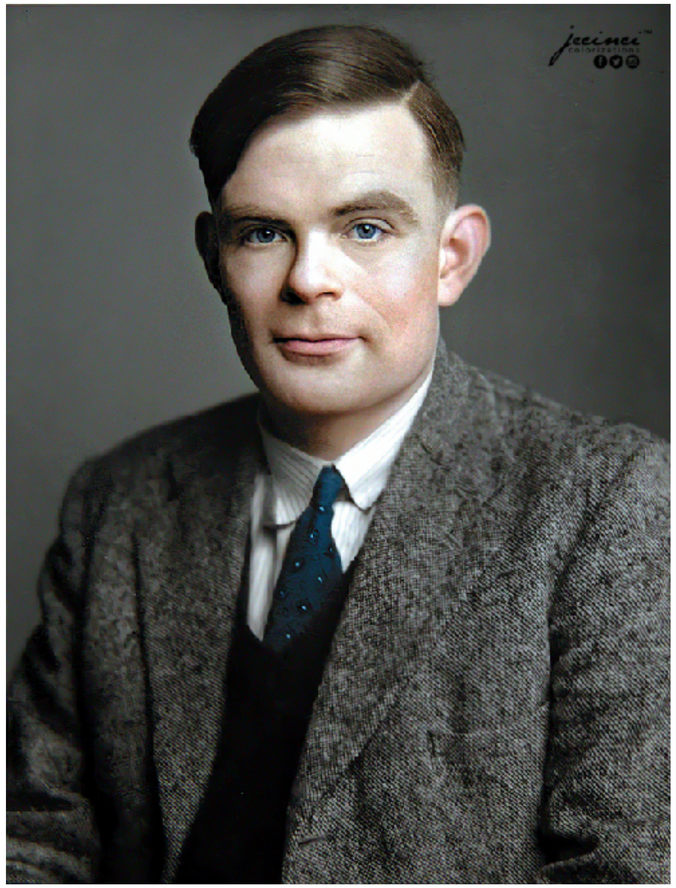
On Computable Numbers, with an Application to the Entscheidungsproblem

What is a Computable Number?

$$0.d_1 d_2 d_3 \cdots$$

Let $\Sigma = \{0,1,2,\ldots,9\}$

Then $[0,1) \cong \Sigma^\omega$.



Alan Turing

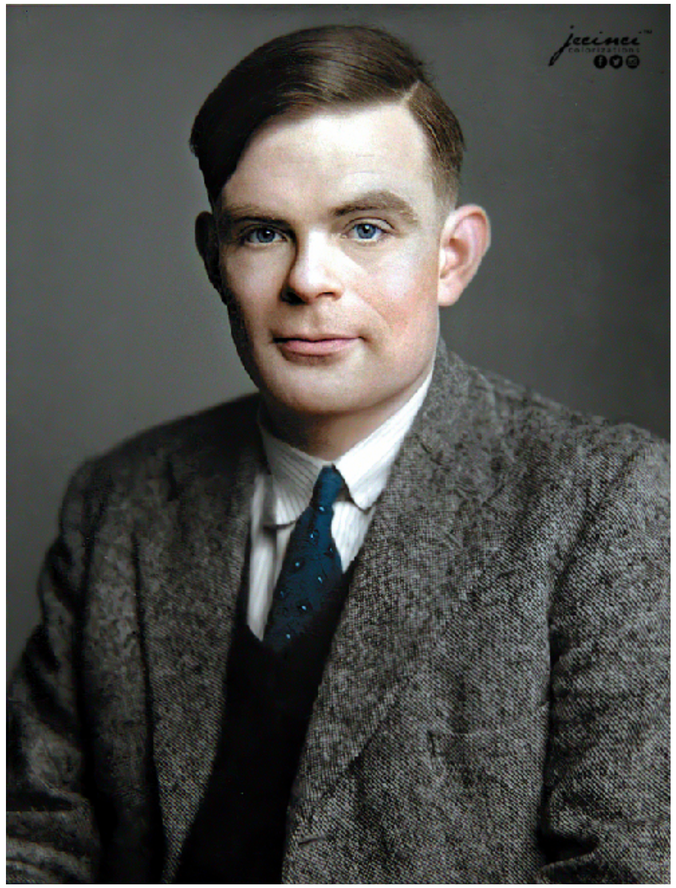
On Computable Numbers, with an Application to the Entscheidungsproblem

What is a Computable Number?

$$0.d_1 d_2 d_3 \cdots$$

Let $\Sigma = \{0,1\}$

Then $[0,1) \cong \Sigma^{\omega}$.



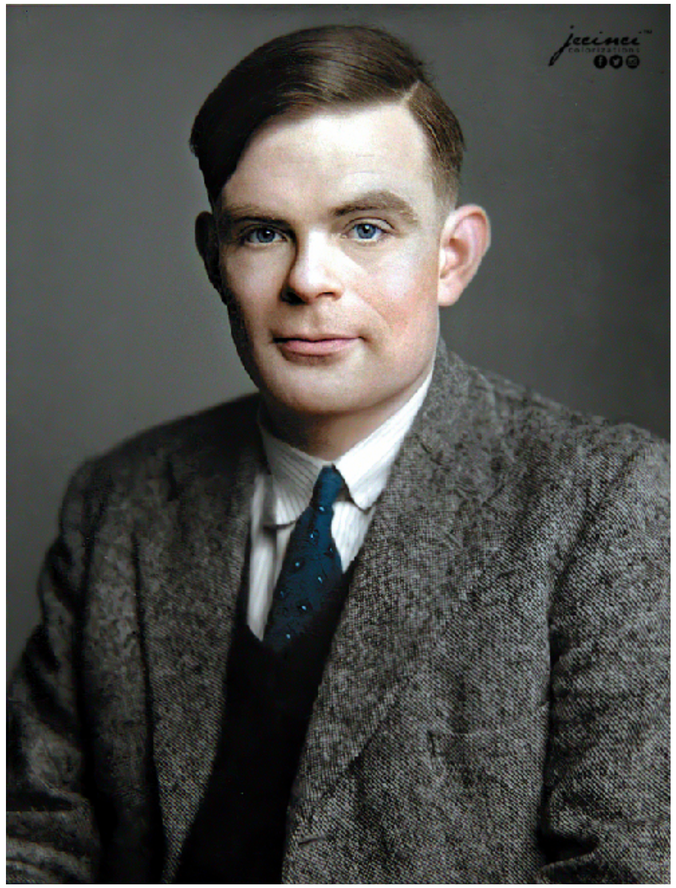
Alan Turing

On Computable Numbers, with an Application to the Entscheidungsproblem

What is a Computable Number?

Equivalent formulation:

A number is computable, if there exists a complete (halting) Turing Machine such that given $n \in \mathbb{N}$, n is accepted if the nth digit is 1 and rejected otherwise.



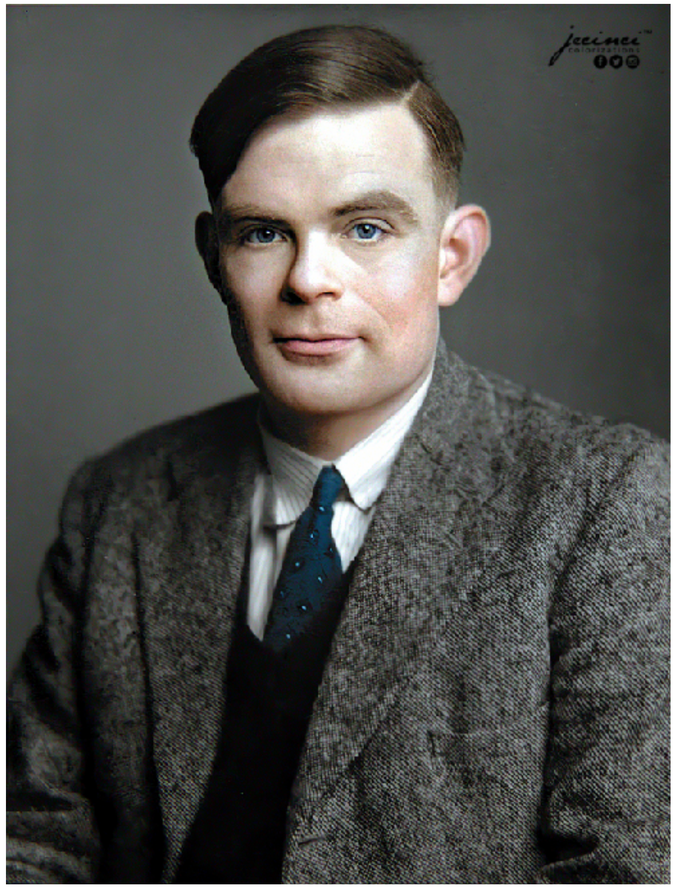
Alan Turing

On Computable Numbers, with an Application to the Entscheidungsproblem

What is a Computable Number?

Every rational number is computable by a DFA.

What about $\sqrt{2}$?



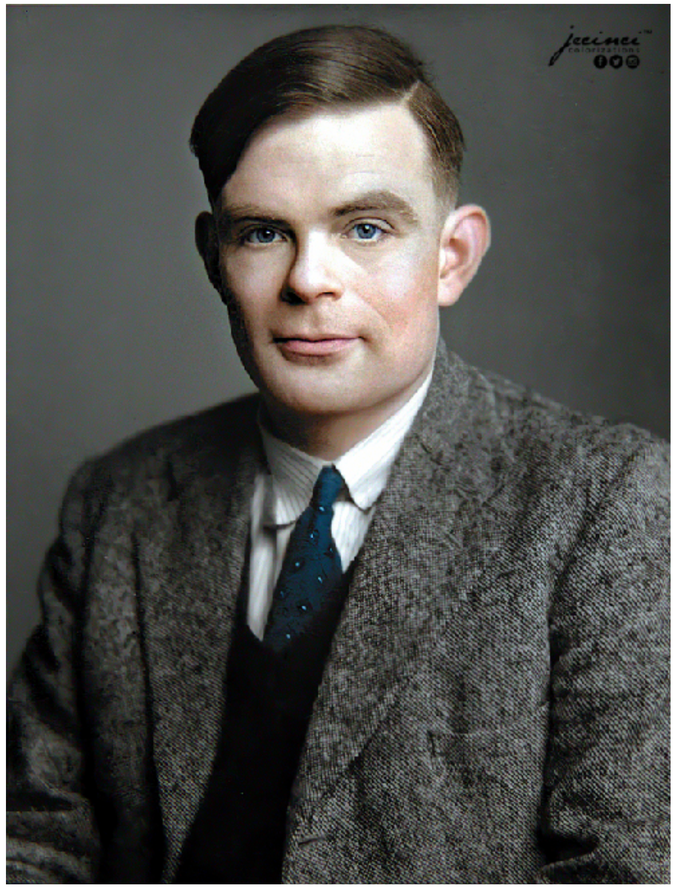
Alan Turing

On Computable Numbers, with an Application to the Entscheidungsproblem

What is a Computable Number?

Every rational number is computable by a DFA.

What about π?



Alan Turing

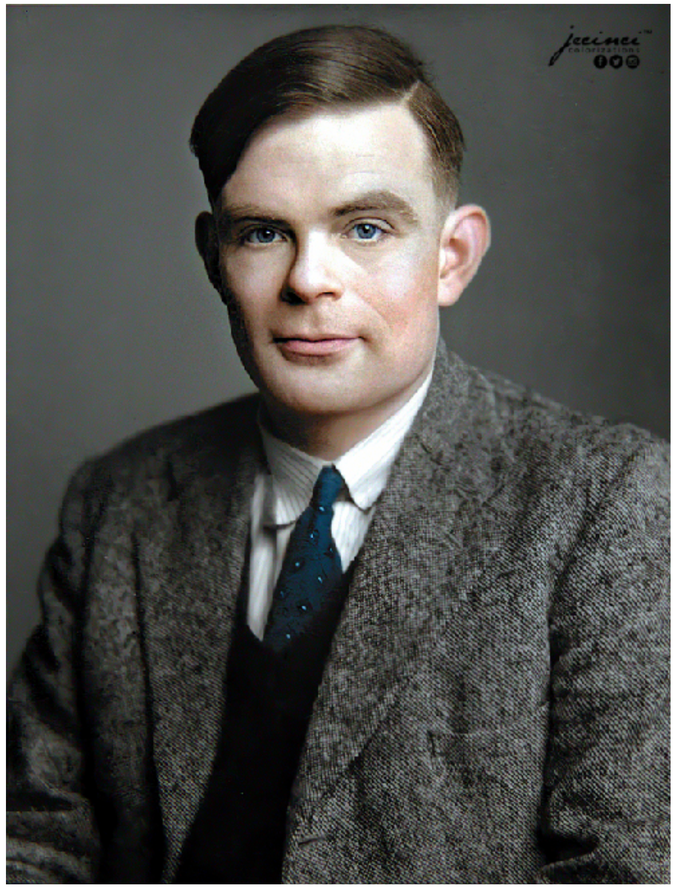
On Computable Numbers, with an Application to the Entscheidungsproblem

What is a Computable Number?

Every rational number is computable by a DFA.

What about π?

Are there non-computable numbers?



Alan Turing

On Computable Numbers, with an Application to the Entscheidungsproblem

Are there non-computable numbers?

Old School Diagonalisation

If a number is not computable, is it *real*?

Busy Beaver Numbers!

Fix a tape alphabet $\Gamma = \{0,1\}$

Let $\mathscr{T}_n$ be the set of all n state Turing machines.

They all start without input with tape full of 0s.

Let $H_n = \{M \in \mathscr{T}_n \mid M \text{ halts}\}$.

Let $\sigma(M)$ be the be the number of steps taken by M when it halts.

Busy Beaver Numbers!

Let $\mathcal{T}_n$ be the set of all n state Turing machines.

Let $H_n = \{M \in \mathcal{T}_n \mid M \text{ halts}\}$.

Let $\sigma(M)$ be the be the number of steps taken by M when it halts.

$$BB(n) = \max_{M \in H_n} \sigma(M)$$

Tibor Radó. *On Non-Computable Functions*. 1962.

Busy Beaver Numbers!

Let $s = 0.1^{BB(1)}01^{BB(2)}01^{BB(3)}0\ldots$

This number is not computable!

Busy Beaver Numbers!

Let $s = 0.1^{BB(1)}01^{BB(2)}01^{BB(3)}0\ldots$

For sake of contradiction, suppose s was computable.

That is we can count $BB(n)$ for any n.

Busy Beaver Numbers!

Let $s = 0.1^{BB(1)}01^{BB(2)}01^{BB(3)}0\ldots$

For sake of contradiction, suppose s was computable.

That is we can count $BB(n)$ for any n.

Given M and w, suppose M has n states.

Busy Beaver Numbers!

Let $s = 0.1^{BB(1)}01^{BB(2)}01^{BB(3)}0\ldots$

For sake of contradiction, suppose s was computable.

That is we can count $BB(n)$ for any n.

Given M and w, suppose M has n states.

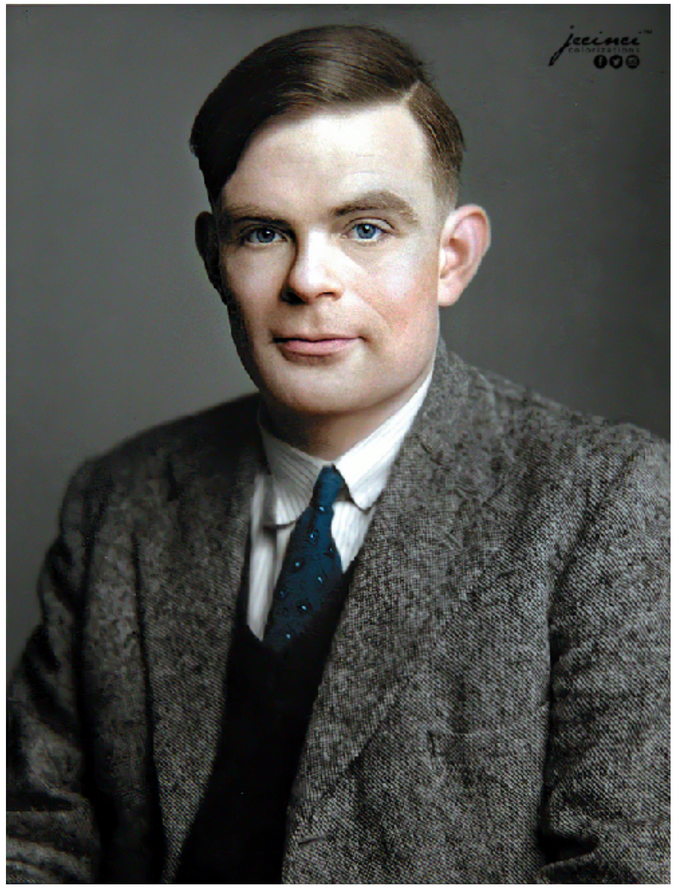
Simulate M on w for $BB(n)$ steps. If it halts, you are done. Else it does not halt!

Busy Beaver Numbers!

Let $s = 0.1^{BB(1)}01^{BB(2)}01^{BB(3)}0\ldots$

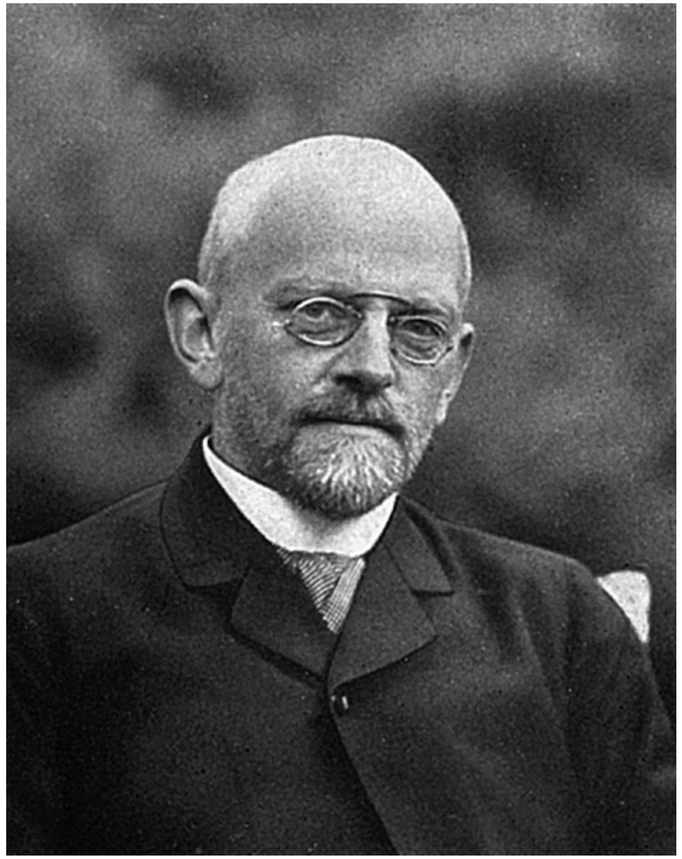
s is computable $\iff$ A_{TM} is recursive

What is a Entsomethingproblem?



Alan Turing

On Computable Numbers, with an Application to the Entscheidungsproblem



David Hilbert

Entscheidungsproblem

(1928)

Is there a mechanical procedure that decides the validity of statements in *First-Order Logic*.

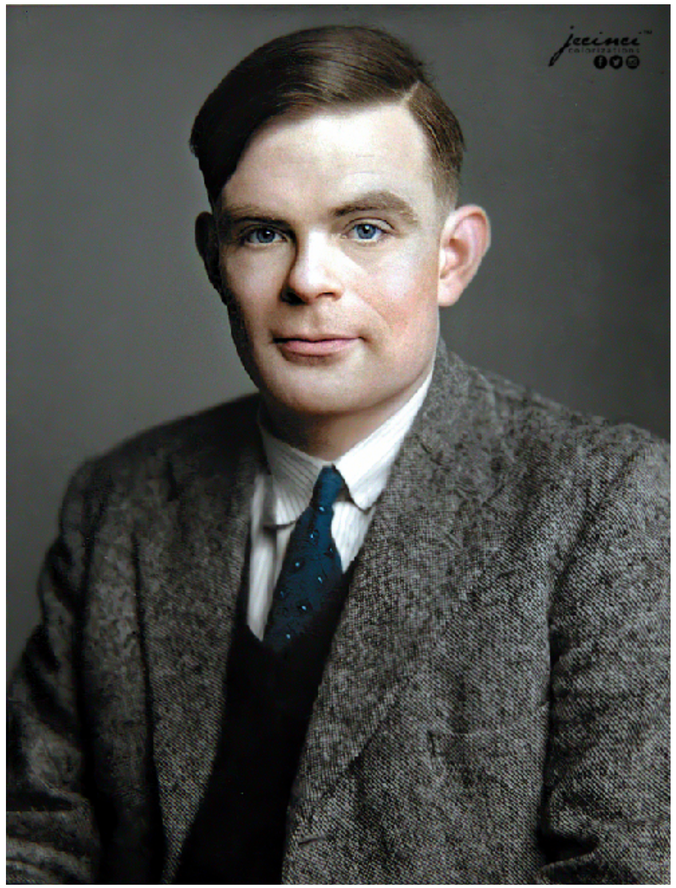
Wilhelm Ackermann

Alonzo Church

A Note on the Entscheidungsproblem

Entscheidungsproblem

No.



Alan Turing

On Computable Numbers, with an Application to the Entscheidungsproblem

Propositional Logic

Let $\mathscr{P} = \{A, B, C, D\}$ and $\{ \neg, \vee , \wedge \}$

Semantics defined by a truth table.
Interpretation is recursive.

A formula φ is said to be valid if for all assignments w, $w \vDash \varphi$.

Is validity decidable?

First-Order Logic

Fix the following:

1. Logical connectives
2. Quantifiers
3. Constant symbols
4. Function and relation symbols (including equality)

First-Order Logic

The following are terms:

1. All constant symbols
2. All variable symbols
3. If $t_1, t_2, \ldots, t_n$ are terms then so is $f(t_1, t_2, \ldots, t_n)$
4. ... and nothing more.

First-Order Logic

The following are WFF:

1. If $t_1, t_2, \ldots, t_n$ are terms then $R(t_1, t_2, \ldots, t_n)$ is WFF
2. Boolean combination of well-formed formulae
3. Quantification of well-formed formulae
4. ... and nothing more.

First-Order Logic

Define semantics as follows:

1. For terms, use the interpretation function I
2. For logical connectives, use the same as propositional logic
3. For quantifiers,

$M, \sigma \vDash \forall x \,.\, \phi(x)$ if for every $d \in D$, such that $M, \sigma[x \mapsto d] \vDash \phi(x)$

$M, \sigma \vDash \exists x \,.\, \phi(x)$ if there exists $d \in D$, such that $M, \sigma[x \mapsto d] \vDash \phi(x)$

First-Order Logic

A formula φ is said to be valid if for all models M, $M \vDash \varphi$.

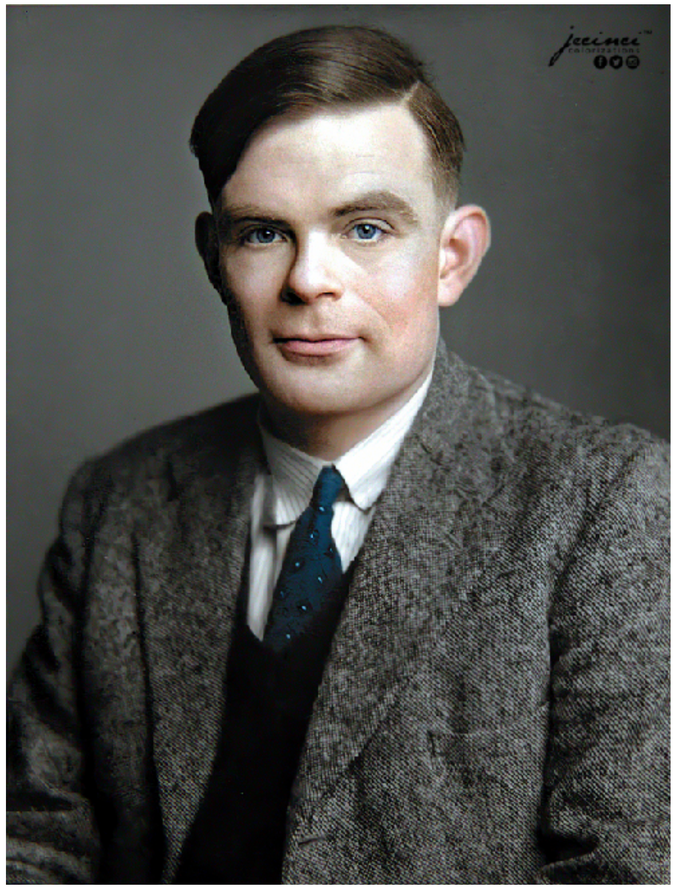
Is validity decidable?

Alonzo Church

A Note on the Entscheidungsproblem

Entscheidungsproblem

No.



Alan Turing

On Computable Numbers, with an Application to the Entscheidungsproblem

Undecidability of Entscheidungsproblem

For sake of contradiction, suppose there was a Turing machine B that decides:

$$\mathrm{FOL}_{\mathrm{VALID}} = \{\phi \mid \phi \text{ is a valid first order logic formula}\}$$

Undecidability of Entscheidungsproblem

For sake of contradiction, suppose there was a Turing machine B that decides:

$$\mathrm{FOL}_{\mathrm{VALID}} = \{\phi \mid \phi \text{ is a valid first order logic formula}\}$$

We will construct a machine to decide A_{TM}.

Undecidability of Entscheidungsproblem

Given a Turing machine M and string w,

Consider the constant symbol ε, denoting empty string.

For each letter $\gamma \in \Gamma$, consider a unary function $\gamma : w \to w$.

So $11/3$ can be represented as $1(1(/(3(\varepsilon))))$.

Undecidability of Entscheidungsproblem

Given a Turing machine M and string w,

Consider the constant symbol ε, denoting empty string.

For each letter $\gamma \in \Gamma$, consider a unary function $\gamma : w \to w$.

For each state $q \in Q$, consider a binary predicate $R_q \subseteq w \times w$.

Undecidability of Entscheidungsproblem

For each letter $\gamma \in \Gamma$, consider a unary function $\gamma : w \to w$.

For each state $q \in Q$, consider a binary predicate $R_q \subseteq w \times w$.

Whenever M has a transition from state q to state q', reading a, writing b, and moving right:

$$\forall x \,.\, \forall y \,.\, R_q(x, ay) \to R_{q'}(xb, y)$$

holds.

Undecidability of Entscheidungsproblem

For each letter $\gamma \in \Gamma$, consider a unary function $\gamma : w \to w$.

For each state $q \in Q$, consider a binary predicate $R_q \subseteq w \times w$.

Whenever M has a transition from state q to state q', reading a, writing b, and moving left:

$$\forall x \,.\, \forall y \,.\, R_q(xc, ay) \to R_{q'}(x, cby)$$

holds for all c.

Undecidability of Entscheidungsproblem

For each letter $\gamma \in \Gamma$, consider a unary function $\gamma : w \to w$.

For each state $q \in Q$, consider a binary predicate $R_q \subseteq w \times w$.

Additionally:

$$\forall y \,.\, R_q(\varepsilon, ay) \to R_{q'}(\varepsilon, by)$$

and others when $y = \varepsilon$.

Undecidability of Entscheidungsproblem

For each letter $\gamma \in \Gamma$, consider a unary function $\gamma : w \to w$.

For each state $q \in Q$, consider a binary predicate $R_q \subseteq w \times w$.

Let Λ be the conjunction of all these formulae.

$$R_{q_0}(\varepsilon, w) \wedge \Lambda \to \exists x \,.\, \exists y \,.\, R_{q_a}(x, y)$$

If and only if M accepts w.

Undecidability of Entscheidungsproblem

For each letter $\gamma \in \Gamma$, consider a unary function $\gamma : w \to w$.

For each state $q \in Q$, consider a binary predicate $R_q \subseteq w \times w$.

$$\phi_{M,w} = R_{q_0}(\varepsilon, w) \wedge \Lambda \to \exists x \,.\, \exists y \,.\, R_{q_{\mathtt{a}}}(x, y)$$

Undecidability of Entscheidungsproblem

For each letter $\gamma \in \Gamma$, consider a unary function $\gamma : w \to w$.

For each state $q \in Q$, consider a binary predicate $R_q \subseteq w \times w$.

$$\phi_{M,w} = R_{q_0}(\varepsilon, w) \wedge \Lambda \to \exists x \,.\, \exists y \,.\, R_{q_{\mathtt{a}}}(x, y)$$

$$\phi_{M,w} \in \mathrm{FOL}_{\mathrm{VALID}} \iff \langle M, w \rangle \in A_{\mathrm{TM}}$$

Undecidability of Entscheidungsproblem

$$\phi_{M,w} \in \mathrm{FOL}_{\mathrm{VALID}} \iff \langle M, w \rangle \in A_{\mathrm{TM}}$$

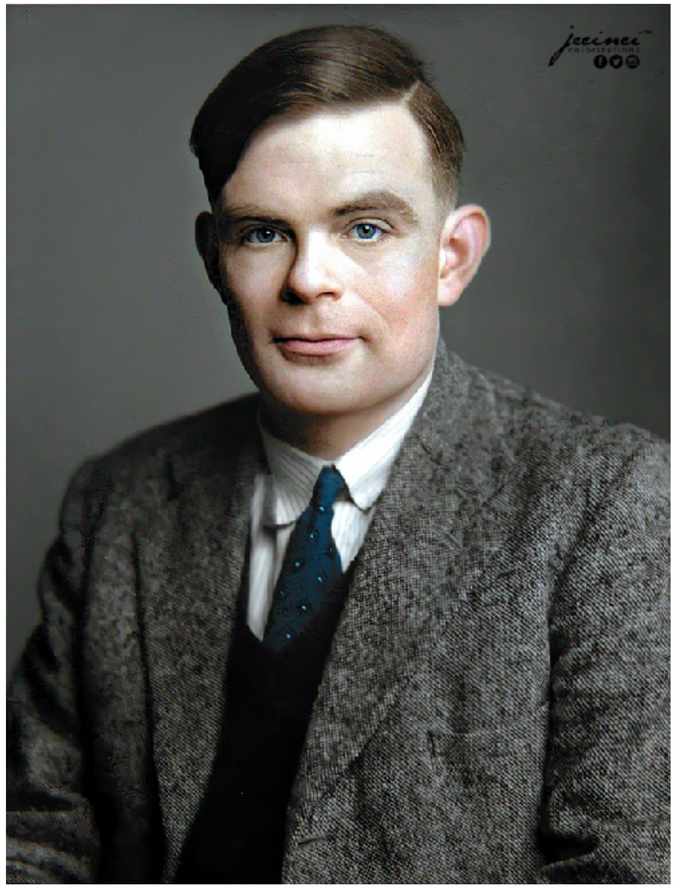
Given a machine B for $\mathrm{FOL}_{\mathrm{VALID}}$, we can design a machine that takes $\langle M, w \rangle$ that first constructs $\phi_{M,w}$ and then runs B on it.

Therefore, membership in $\mathrm{FOL}_{\mathrm{VALID}}$ is undecidable.

Computation and Its Limitations!

The following are not computable:

A_{TM}, $\mathrm{ALL}_{\mathrm{CFL}}$, $\mathrm{HALT}_{\mathrm{TM}}$, $BB(n)$, $\mathrm{FOL}_{\mathrm{VALID}}$.



Alan Turing

On Computable Numbers, with an Application to the Entscheidungsproblem